



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 8

TEMPERATURE MEASUREMENT AND CONTROL

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 8

Temperature Control System

Objectives

1. To design and build a signal-conditioning circuit for an NTC thermistor.
2. To interface the conditioned temperature signal to the PC via the PCIE-1750-U and ADAM-3937.
3. To implement a computer-based system that displays the measured temperature and performs ON/OFF control of an indicator LED on the trainer.

Equipment / Software Required

- NTC thermistor and trainer board with indicator LED
- Components for the student-designed signal-conditioning circuit (op-amps, resistors, capacitors)
- ADAM-3937 DB37 wiring terminal board
- PCIE-1750-U isolated digital I/O card, installed in the host PC
- PC with a C/C++ or Python development environment
- Digital multimeter

Introduction

In many industrial processes, temperature must be regulated automatically. Two general strategies are used: *logic (ON/OFF) control*, in which a heater, cooler, or indicator is simply switched on or off when a threshold is crossed, and *fuzzy/proportional control*, which requires a more elaborate definition of “hot” or “cold” before acting. This experiment uses logic control.

An NTC (negative temperature coefficient) thermistor is a resistor whose resistance decreases as its temperature increases. Given the thermistor’s resistance–temperature characteristic (from its datasheet), a measured resistance can be converted to a temperature, and vice versa.

The signal path for this experiment is: the thermistor’s resistance is converted to a voltage by a signal-conditioning circuit (which the student must design), this voltage is digitised and read by the PC via the PCIE-1750-U (through the ADAM-3937 wiring terminal), and software on the PC computes the corresponding temperature, displays it, and drives the indicator LED on the trainer according to a threshold comparison (Figure 8.1).

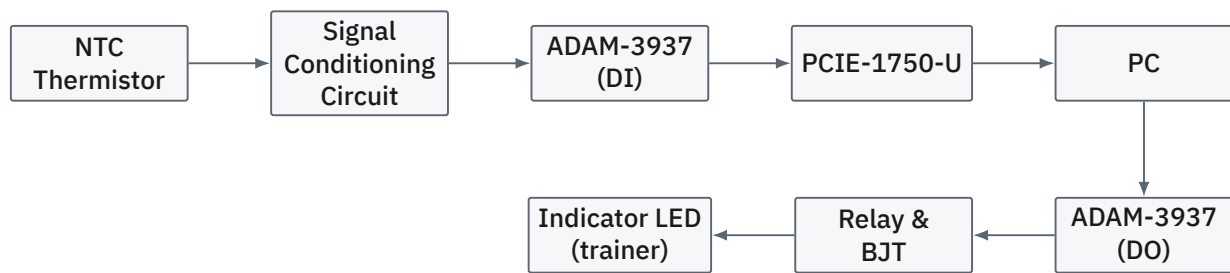


Figure 8.1: Block diagram of the temperature measurement and control system.

Procedure

Activity 1 – Signal-conditioning circuit design

Objective: design and build a circuit that converts the thermistor's resistance to a voltage suitable for the PCIE-1750-U input.

1. Obtain the resistance–temperature characteristic for the NTC thermistor from its datasheet, and determine the resistance range corresponding to the temperature range you intend to measure and control.
2. Design a signal-conditioning circuit that converts the thermistor's resistance to a voltage. Calculate your component values and justify your design choices, including the expected output voltage range over your chosen temperature range.
3. Include a voltage-follower (buffer) stage to eliminate loading effects between the conditioning circuit and the downstream measurement hardware.
4. Build the circuit and verify its output voltage against a reference temperature measurement at two or more points, and check linearity/consistency with your design calculations.

Activity 2 – Interfacing to the DAQ hardware

Objective: connect the conditioned temperature signal and the LED control line to the PC via the ADAM-3937 and PCIE-1750-U.

1. Connect the output of your signal-conditioning circuit to an input channel on the ADAM-3937 terminal board.
2. Connect the trainer's indicator LED control line to an isolated digital output channel on the ADAM-3937 via BJT/Relay circuit.
3. Connect the ADAM-3937 to the PCIE-1750-U card installed in the host PC via the supplied cable.
4. Verify both the input and output wiring using the DAQ card's bundled configuration/monitoring utility before writing any code: confirm the conditioned voltage reads correctly, and confirm the LED can be toggled from the utility.

Activity 3 – Temperature monitoring and LED control software

Objective: implement software that displays the measured temperature and controls the LED according to a threshold.

1. In C, C++, or Python, write a program that:
 - Periodically reads the conditioned thermistor voltage,
 - Converts the reading to a temperature using your circuit's calibration from Activity 1,
 - Displays the current temperature in a simple GUI, updating in real time, and
 - compares the measured temperature against the threshold and sets the LED outputs accordingly.
2. Report and justify your design choice for signal conditioning circuit.
3. Use the flowchart in Figure 8.2 as a starting point for your control logic, adapting the ON/OFF branch to match your chosen design.

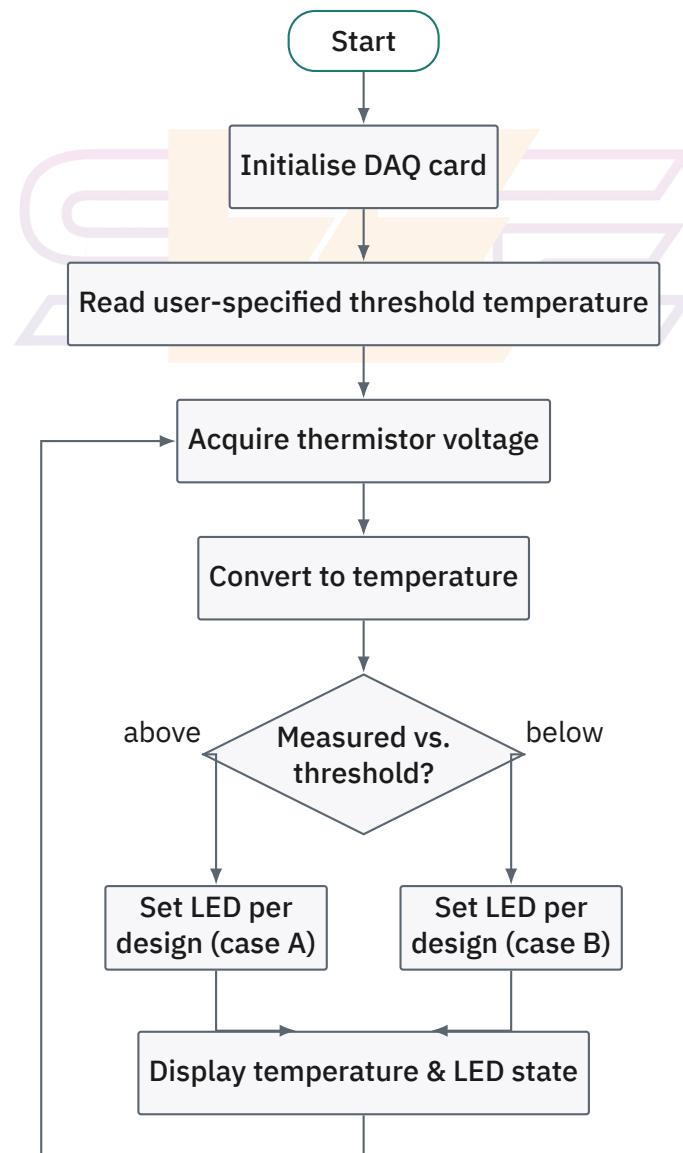


Figure 8.2: Temperature monitoring and LED control loop (ON/OFF direction per student design).

4. Set the controlled (threshold) temperature to a chosen value and record the measured temperature and LED state at 1-minute intervals for 35 minutes (Table 8.1). Plot temperature against time.
5. Repeat for at least two further threshold temperatures. Plot your results.
6. Discuss your findings, including the responsiveness of the LED control and the accuracy of your temperature conversion.

Table 8.1: Temperature and LED-state log.

Threshold temp.	Time (min)	Measured temp.	LED state
	1		
	2		
	...		
	35		

Report

1. Present your signal-conditioning circuit design (topology, calculations, component values) and its calibration against a reference temperature measurement.
2. State and justify your chosen LED ON/OFF logic (Activity 3, step 2).
3. Present and discuss your temperature-vs-time plots for each threshold tested.
4. What sources of error or lag affect the accuracy and responsiveness of this system (thermistor thermal mass, signal-conditioning bandwidth, software polling rate, etc.)?